

RISC-V register details:

→ We could store any information in any register

If certain registers are used for certain purposes then it will increase readability.

11 ⇒ Saved register

7 ⇒ Temporary register.

Register Num	Functionality
x0	holds constant value 0.
x1	to store return address that you should return to after a function call.
x2	stack pointer → stores an address in memory with the top of the stack
x3	Global pointer
x4	Thread pointer
x5-7	Temporaries
x8	Frame pointer
x9	Saved register ⇒ To store variables
x10-x11	Function arguments / return value
x12-x17	Function arguments.
x18-27	Saved registers
x28-31	Temporaries. for holding temp values

for values passed to a function and for values return to a function

$$(a+b) - (c+d)$$

Operands: part of instruction that contains the data to acted on on, the memory location of the data in a register.

Based on physical location—

- ↳ Register operand
- ↳ Memory operand
- ↳ Constant

add x_{28}, x_{29}, x_{30}
 $\nearrow \quad \sqcup$

Arithmetic instructions use register operands.

Suppose:

$$f = (g+h) - (i+j);$$

↳ Only one operation is performed per

RISC-V instructions.

Code:

add x_5, x_{20}, x_{21}

add x_6, x_{22}, x_{23}

sub x_{19}, x_5, x_6

$\uparrow \quad \uparrow$
 $(g+h) \quad (i+j)$

$g+h$ is a step toward our actual answer.

1 code line divided into multiple instructions.

$$f = x_{19}$$

$$g \Rightarrow x_{20}$$

$$h \Rightarrow x_{21}$$

$$i \Rightarrow x_{22}$$

$$j \Rightarrow x_{23}$$

Add/Sub instruction syntax:

add , ,
 dest source1 source2

$$\Rightarrow \text{dest} = \text{source1} + \text{source2}$$

sub , ,
 dest source1 source2

$$\Rightarrow \text{dest} = \text{source1} - \text{source2}$$

Memory operand:

* We store composite data in main memory.

* In order to perform arithmetic operations we must load data.

* Memory is byte addressed.

* No alignment restrictions

↳ words must start at addresses that are multiple of

4

↳ double words must start at addresses that are multiple of

8

Each slot in memory can store 8 bits of data

$A \Rightarrow \text{length} = 3$

$A[0], A[1], A[2]$

0000 $\rightarrow$ 0007 $\rightarrow A[0]$

0008 $\rightarrow$ 0015 $\rightarrow A[1]$

0016 $\rightarrow$ 0023 $\rightarrow A[2]$

$A[3] = ?$ [find out physical address]

$8 \times \text{index}$

$\rightarrow 64 \text{ bit}$

offset + base

$8 \times 3 + 0000$

$= 0024$

32 bit $\Rightarrow$ loc # 0004

64 bit $\Rightarrow$ loc # 0008

Hence in order to retrieve a 64 bit data we need to choose 8 consecutive slots.

Endianness

The order in which computer memory stores data. (byte)

↳ Big Endian

↳ Little Endian

↳ Bytes are ordered from right to left

↳ LSB stored at lowest address

# 0000	B7	0000	1010
# 0001	B6	0000	0000
# 0002	B5	0000	0000
# 0003	B4	0000	0000
# 0004	B3	0000	0000
# 0005	B2	0000	0000
# 0006	B1	0000	0000
# 0007	B0	0000	0000

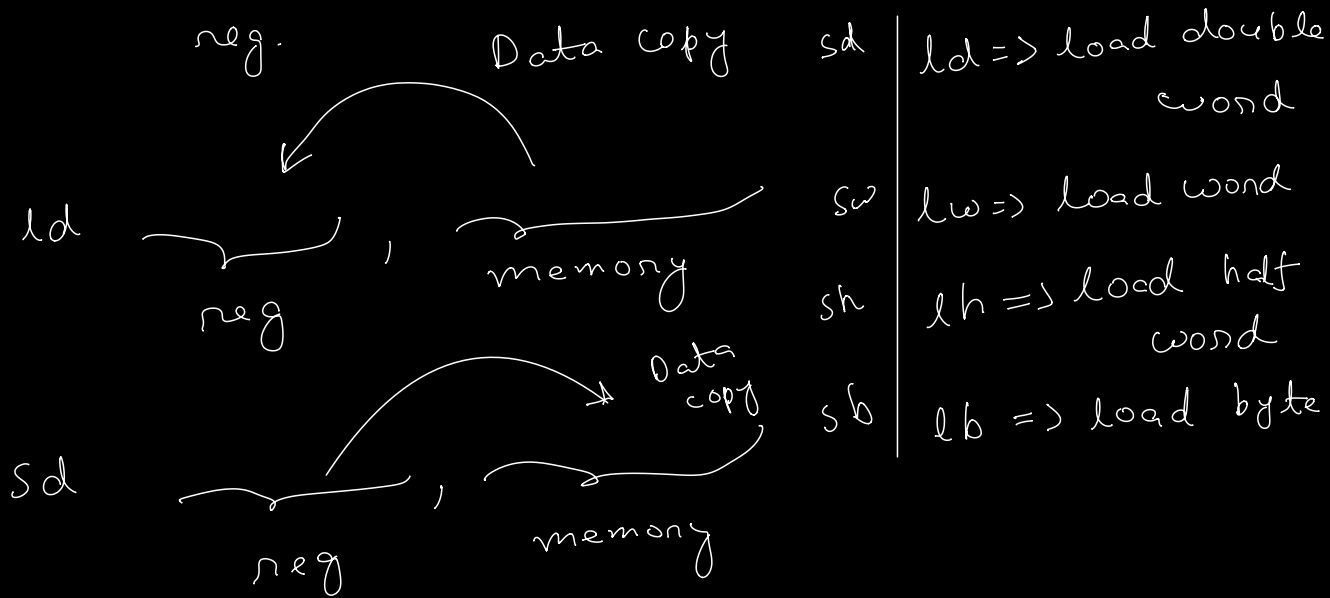
If we perform add:

$$\begin{array}{r}
 1011 \quad 1100 \quad 1100 \quad 1101 \\
 + 1 \\
 \hline
 1011 \quad 1100 \quad 1100 \quad 1110
 \end{array}$$

Load/Store syntax:

Load = loads data from memory to reg.

Store = stores data back to memory from



ld X_{21} , 64 [X_{20}]
 reg des 4 reg source

$$X_{20} = 0000$$

$$0000 + 64 = 0064$$

sd X_{21} , 40 [X_{20}]
 reg source 4 reg source
 1 2

$$A[8] \Rightarrow A[5]$$

Let's assume that A is an array of 100 doublewords and that the compiler has associated the variables g and h with the registers $x20$ and $x21$ as before. Let's also assume that the starting address, or *base address*, of the array is in $x22$. Compile this C assignment statement:

$g = h + A[8];$

$\uparrow$ memory
 $\text{ld } x5, 64[x22]$
 $\uparrow \quad \uparrow \quad \uparrow$
 Temp offset base reg
 Index = 8
 $A[8] = 8 \times 8 = 64$

$g = x20$

$h = x21$

Base of $A = x22$

$\text{Add } \underbrace{x20}_{\text{dest}}, \underbrace{x21}_{s1}, \underbrace{x5}_{s2}$

Assume variable h is associated with register $x21$ and the base address of the array A is in $x22$. What is the RISC-V assembly code for the C assignment statement below?

$A[12] = h + A[8];$

$\text{store } a \text{ dd}$
 $\text{ld } x5, 64[x22]$
 $12 \times 8 = 96$
 $\text{add } x5, x21, x5$
 $\text{sd } x5, 96[x22]$

$h = x21$

Base of $A = x22$

[N.B. destination is $x5$ as now
 I have to store in memory]

Immediate operands: (avoid load instructions)

- * Constant data specified in an instruction.

Add instruction syntax:

$$\text{addi} \quad \underline{\hspace{2cm}}, \quad \underline{\hspace{2cm}}, \quad \underline{\hspace{2cm}}$$

dest source1 source2
(constant)

$$\Rightarrow \text{dest} = \text{source 1} + \text{source 2}.$$

* Compiler cannot add or sub two integers.

Addi $x_{22}, x_{22}, 1$

we can also subtract $\rightarrow$ Addi $x_{22}, x_{22}, -5$

Sign Expression

representing a number with more bits.

Keep the same value same

↳ if signed :

Replicate the sign bit to the left.

else:

extend 0s to the left.

LB $\Rightarrow$ load Byte (Signed Extension)

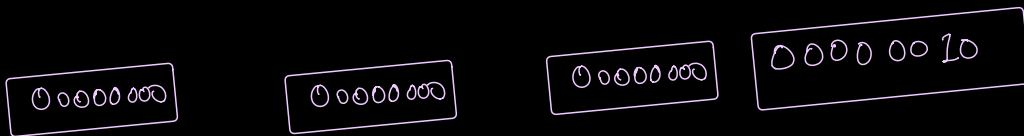
LBU $\Rightarrow$ Load Byte (Unsigned extension)

$$\# (2)_{10} = (10)_2$$

2 in 8 bits unsigned

$$= (00000010)_2$$

LBU $x_{22}, 10[x_{21}]$



If

LB

$$(-2)_{10} \Rightarrow 010$$

$$\Rightarrow 00000 \quad 010$$

$$(-2)_{10} \Rightarrow \begin{array}{r} 11111101 \\ +1 \end{array}$$

$$\begin{array}{r} 11111101 \\ +1 \\ \hline (11111110)_{2s} \end{array}$$

0 to $2^n - 1 \Rightarrow$ unsigned

-2^{n-1} to $(2^{n-1}) - 1 \Rightarrow$ signed [2s]

Translating a RISC-V assembly instruction into a machine instruction.

$$A[30] = h + A[30] + 1;$$

How this is executed.

ld x9, 240(x10) // Temporary reg x9 gets A[30]
add x9, x21, x9 // Temporary reg x9 gets h + A[30]
addi x9, x9, 1 // Temporary reg x9 gets h + A[30] + 1
sd x9, 240(x10) // Stores h + A[30] + 1 back into A[30]

Higher level
language

↓ Compiler

Assembly Lang

↓ Assembler

Machine language
programme.

Machine only understands high and low

In RISC-V instruction takes exactly 32 bits
(one word)

The layout of the instruction is called
instruction format.

Divide 32 bits of an instruction into "fields"

↳ conflict

Desire to keep all instructions length same.

vs.

Desire to have a single instruction format.

Suppose

Add X_{20}, X_{20}, X_{21}
 $\underbrace{\hspace{1cm}}_{rd} \quad \underbrace{\hspace{1cm}}_{rs1} \quad \underbrace{\hspace{1cm}}_{rs2}$

Addi $X_{20}, X_{20}, 5$
 $\underbrace{\hspace{1cm}}_{rd} \quad \underbrace{\hspace{1cm}}_{rs} \quad \underbrace{\hspace{1cm}}_{\text{immediate}}$

ld $X_{20}, 16[X_{21}]$
 $\underbrace{\hspace{1cm}}_{rd} \quad \underbrace{\hspace{1cm}}_{\text{immediate}} \quad \underbrace{\hspace{1cm}}_{rs}$

sd $X_{20}, 16[X_{21}]$
 $\underbrace{\hspace{1cm}}_{rs-2} \quad \underbrace{\hspace{1cm}}_{\text{immediate}} \quad \underbrace{\hspace{1cm}}_{rs-1}$

Instruction Formats:

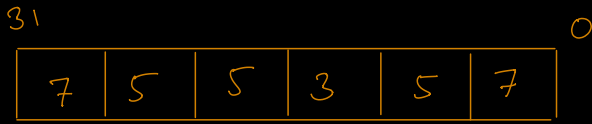
↳ R type = instructions that use 3 registers
 (Add, Sub, SLL, XOR, OR,
 AND)

↳ I type = instructions that use immediate
 and 2 registers (Addi, SLLI, SRLI, ORI, ANDI,
 Load)

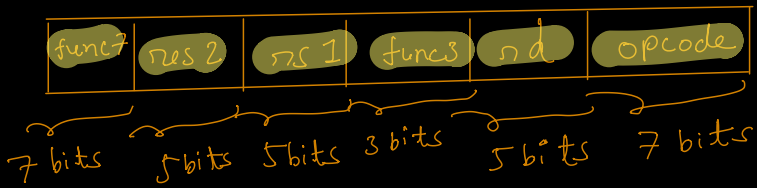
↳ S type = store information.

R-Type Instruction:

→ exactly 3 registers must
 ↓
 1 ⇒ rd
 2 ⇒ rs



each field has unique name and size.



Opcode: Partially denotes the format of an instruction and instruction itself.

rd: register destination operand

rs1: register source 1 operand

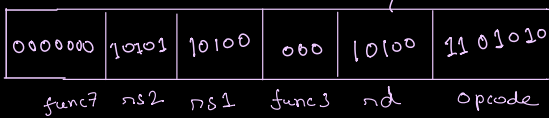
rs2: register source 2 operand

function 3: }
 function 7: } → their combination tells us which instruction to perform.

Example:

015A0A6A

Only reg no binary not the content

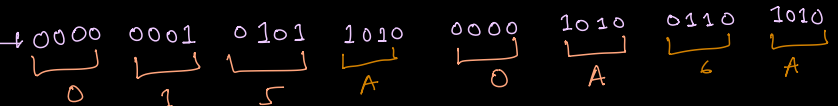


If not given inst
 0000000 [bin value]

Add X20, X20, X21
 rd rs rs2

Hint can be given in question regarding Opcode
 First figure out which type of instruction it is.

Hexa

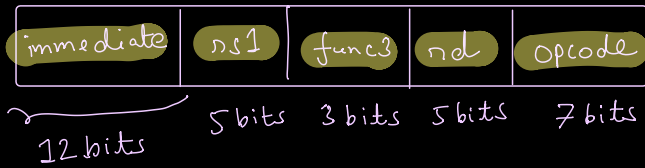


I-type Instructions

31



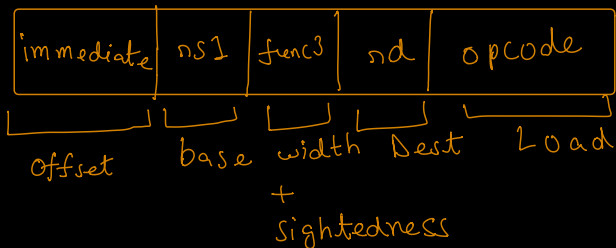
each field has unique name and size:



immediate = constant / offset [2's complement]

rs1 = Source / Base register number.

Load



func3 [size of the data that is loaded]

Also determine if it is signed or unsigned

Suppose ques may come func3 → 6 is the number that represent signness of load instructions.

From table respective binary number I have to find if it is LB / LH / LW / LD.

①

one immediate val

②

Two registers involve

Lo rd

Lo rs

Add: X22, X22, 5

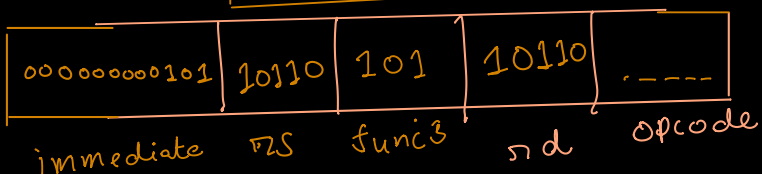
↑
const

ld: X22, 16[X21]

↑ ↗
offset rs1

LB
LH
LW
LD

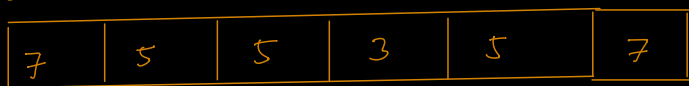
Example: Add X22, X22, 5



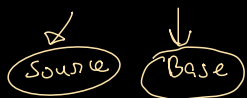
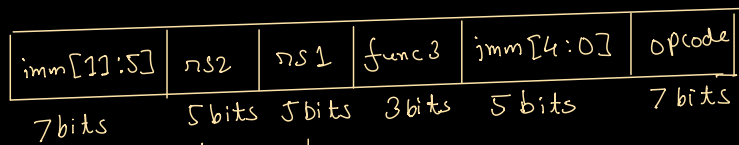
S type instruction:

31

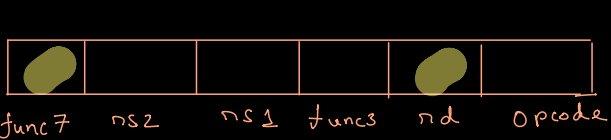
0



each field has unique name and size.

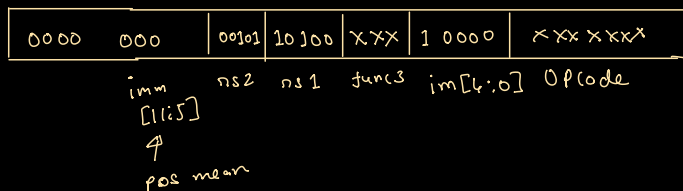
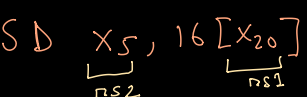
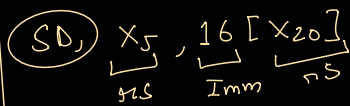
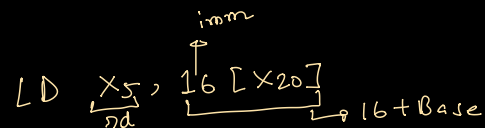


reason for imm split is they want to keep rs1 and rs2 fields in the same place in all instruction formats



=> R - match with

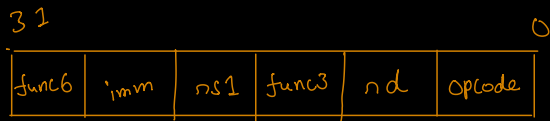
We divide 12 bits -> 2 parts [7 and 5]
 In R type func7 was 7 bit and rd was 5 bit.
 That's why for matching we distinguish into 7 and 5 bits. Just
 in S7 I don't have dest so we split immediate value.



Shift Operations

↳ follows I type but modified.

Slli $X_{11}, X_{19}, 4$
 $\underbrace{\hspace{1cm}}_{rd} \quad \underbrace{\hspace{1cm}}_{rs} \quad \underbrace{4}_{imm}$



↳ immediate broken down into 2 fields.

Why?? $\Rightarrow$ If you shift a 64 bit value more than 63 bits what happens?

Shift left

↳ shift left and fill the positions with 0.

$\Rightarrow$ We can perform multiplication by 2^i using slli.

Slli $X_{11}, X_{19}, 4$



the value that will be stored in X_{11} is basically, val in $X_{19} \times 2^4$

Shift Right:

↳ shift right and fill the positions with 0.

$\Rightarrow$ We can perform division by 2^i using srli

Srli $X_{11}, X_{19}, 4$



the value will be stored in X_{11} is basically, val in $X_{19} / 2^4$

SLLI $\rightarrow$ Shift Left Logical Immediate.

1010 1111

1011 1100 $\leftarrow$ 2 time left shift



0101 1110 $\leftarrow$ 1 time right shift

Slli $\rightarrow$ Shift left logical i

Slli $X_{19}, X_{20}, 4$
 $\underbrace{\hspace{1cm}}_{rd} \quad \underbrace{\hspace{1cm}}_{rs1} \quad \underbrace{4}_{imm \text{ const}}$

0000 1100 = 12

0001 1000 = 24 $\Rightarrow 12 \times 2^1$

0011 0000 $\Rightarrow 48 \Rightarrow 12 \times 2^2$

12×7
 $\rightarrow (12 \times 8) - 12$

0011 0000 $\Rightarrow 48$

0001 1000 $\Rightarrow 24 \rightarrow 48 / 2^1$

0000 1100 $\Rightarrow 12 \rightarrow \frac{48}{2^2}$

$$= \frac{\text{number}}{2^i}$$

AND $\rightarrow$ Select some bit and clear others to 0

$\downarrow$
Bit Masking.

and X_9, X_{10}, X_{11}
 $\underbrace{\hspace{1cm}}_{nd} \quad \underbrace{\hspace{1cm}}_{ns_1} \quad \underbrace{\hspace{1cm}}_{ns_2}$

Pattern:

Something and 0 $\rightarrow$ 0

Something and 1 $\rightarrow$ 1

A	B	A.B
0	0	0
1	0	0
0	1	0
1	1	1

$X_{10} = \dots\dots\dots 1001$ Bitwise AND $\rightarrow$ Store in X_9
 $\uparrow\uparrow\uparrow\uparrow$
 $X_{11} = \dots\dots\dots 1100$

OR $\Rightarrow$ Include Bits

OR X_9, X_{10}, X_{11}
 $\underbrace{\hspace{1cm}}_{nd} \quad \underbrace{\hspace{1cm}}_{ns_1} \quad \underbrace{\hspace{1cm}}_{ns_2}$

A	B	A+B
0	0	0
1	0	1
0	1	1
1	1	1

sth + 0 = sth
 sth + 1 = 1

$X_{10} = 0000 \dots\dots 0000 \quad 1100 \quad 0000$
 $X_{11} = 0000 \dots\dots 0000 \quad 0011 \quad 0000$
 $X_9 = 0000 \dots\dots 0000 \quad 0011 \quad 0000$

$\Rightarrow$ You want to set these bits to 1 and rest should remain as it is

XOR $\rightarrow$ Can work as Buffer/Not

XOR X_9, X_{10}, X_{11}
 $\underbrace{\hspace{1cm}}_{nd} \quad \underbrace{\hspace{1cm}}_{ns_1} \quad \underbrace{\hspace{1cm}}_{ns_2}$

A	B	A $\oplus$ B
0	0	0
1	0	1
0	1	1
1	1	0

$\Rightarrow$ XOR with 0 = Buffer

XOR with 1 = NOT

Practise:

Q.

$$A[3] = A[5] + 1$$

Add rd, rs1, rs2
Addi rd, rs1, imm

Ans. double word $\rightarrow 8 \times 5$ (index)

ld x5, 40[x21]
off
addi x6, x5, 1
rs1 imm
sd x6, 24[x21]

if I was told

$A[g]$ instead of $A[5]$

$$\rightarrow A[3] = A[g] + 1$$

$A[g]$

Ans:

Slli x5, x25, 3
9 offset

Add x6, x21, x5

LD x7, 0[x6]

We stored immediate offset at register

Reason

$$1 \rightarrow x \times 2^1$$

$$3 \rightarrow x \times 2^3 = 8$$

$$A[0] = x_{21}$$

A is a double array.

$$A[0] = x_{21}$$

A is a double word array.

$\rightarrow$ Base address of A is stored in x_{21}

N.B

When a value is stored in memory and I have to perform operation

first I have to import val to register from memory then perform add

Again remainder

Double word $\rightarrow 8 \times \text{ind}$

word $\rightarrow 4 \times \text{ind}$

and also told $g \rightarrow x_{25}$

(1) index $\times$ no of memory cells needed to store value

Decision Making

It is commonly represented in programming using the -

(i) If statement.

(ii) goto statement (label)

If $(i == j) :$

Statement 1

else :

Statement 2

assembler runs the code sequentially.

$i == j$ T

stmt-1

stmt-2

RISC-V includes two decision making instructions:
(if statement with a goto)

testing a val

→ $\text{beq } rs1, rs2, L1$
↑ ↑
branch if label
equal

Explanation: Go to the statement labeled "L1" ;

$\text{bne } rs1, rs2, L1$
↑ ↑
branch if not equal to label

Explanation: Go to the statement labeled 'L1' if conditional jump

Conditional jump

	Instruction	Syntax	Operation
=	bneq	bneq rs1, rs2, L1	rs1 == rs2
!=	bne	bne rs1, rs2, L2	rs1 != rs2
<	blt	blt rs1, rs2, L3	rs1 < rs2
>=	bge	bge rs1, rs2, L4	rs1 >= rs2

Q ISC-V code?

Given code,

```

if (i == j) :
    f = g + h
else :
    f = g - h
    
```

```

bne x22, x23, Else
add x19, x20, x21
bneq x0, x0, Exit

Else:
    sub x19, x20, x21

Exit:
    
```

```

f = x19
g = x20
h = x21
i = x22
j = x23
    
```

↳ unconditional branch

Why it good to use

bne :

```

if (i == j) :
    f = g + h
    
```

```

bneq x22, x23, If
If:
    Add x19, x20, x21
Exit:
    
```

```

bne x22, x23, Exit
Add x19, x20, x21
Exit:
    
```


Loop

while (save[i] == K)

$a = a + 1$
 $i = i + 1$

save[2] $\Rightarrow$ index $\times 8$
index $\times 2^3$

slli $x_{10}, x_{22}, 3$] offset

add x_{10}, x_{10}, x_{25}] offset + base

ld $x_9, 0[x_{10}]$

bne x_9, x_{24}, Exit

addi $x_{23}, x_{23}, 1$

addi $x_{22}, x_{22}, 1$

beq x_0, x_0, loop

Avoid
Ambiguity.

$i = x_{22}$

$K = x_{24}$

$a = x_{23}$

save $\Rightarrow x_{25}$

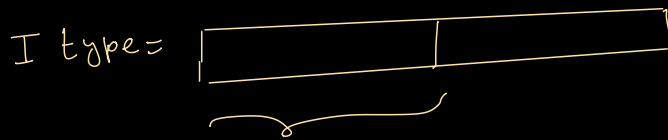
$\nearrow$
Base

save[0]

Exit:

Try this $\Rightarrow X_{19} = (1111\ 1111\ 1111\ 0000)_2$
 $\hookrightarrow 16\text{ bits}$

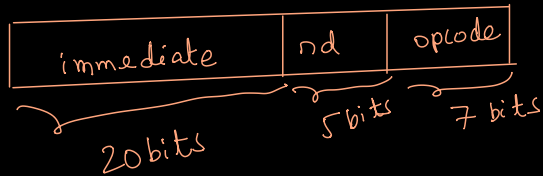
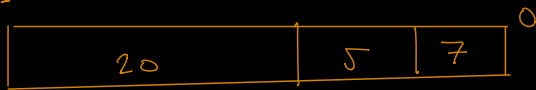
Addi $X_{19}, X_0, 65520$.



then how do we do this?

U type instruction $\rightarrow$ LUI

31



LUI $\rightarrow$ Load Upper Immediate $\rightarrow$ Use it to form 32 bit immediates

Syntax = LUI rd, constant

* Copies the 20 bit data into rd's [31:12]
 * Copy whatever you have in bit 31 to bits

[63:32]

* Copy 0s in [11:0] of rd.

$\rightarrow$ load this 64 bit value into X_{19}

lui $X_{19}, 976$

addi $X_{19}, X_{19}, 1280$

Branching Instructions

SB type instructions $\rightarrow$ beq, bne, blt, bge

31 0

1	6	5	5	3	4	1	7
---	---	---	---	---	---	---	---

* each field has unique name and size

imm [12]	imm [10:5]	rs2	rs1	func3	imm [4:1]	imm [11]	opcode
----------	------------	-----	-----	-------	-----------	----------	--------

* represents branch addresses in multiples of 2.

* forward and backward moving.

* the unusual encoding of imm simplifies datapath design.

slli X10, X22, 3 $\rightarrow$ line 1 #80000

add X10, X10, X25 $\rightarrow$ line 2 #80004

ld X9, 0(X10) $\rightarrow$ line 3 #80008

bne X9, X24, Exit $\rightarrow$ line 4 #80012

addi X22, X22, 1 $\rightarrow$ line 5 #80016

beq X0, X0, loop $\rightarrow$ line 6 #80020

Exit

$\rightarrow$ line 7 #80024

3x4=12

0000 0000 1100
 0 0000 0000 1100 $\rightarrow$ discard it
 12 11 10:5 4:1

0	000000	11000	01001	XXX	0110	0	XXXXXXXX
imm [12]	imm [10:5]	rs2	rs1	func3	imm [4:1]	imm [11]	opcode

1	111111	11000	01001	XXX	0110	1	XXXXXXXX
imm [12]	imm [10:5]	rs2	rs1	func3	imm [4:1]	imm [11]	opcode

5x4=20 but its going upward so -20 $\rightarrow$ see the code

= 0000 0001 0100

PC Relative Addressing = PC + immediate $\times 2$

1	111 111	11000	01001	xxx	0110	1	xxxxxxx
---	---------	-------	-------	-----	------	---	---------

(i) Form the 12 bit number.

11 111 111 0110

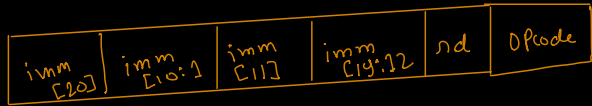
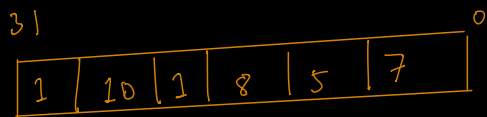
(ii) Detect if positive or negative number.

if neg perform 2's comp again,

$$\begin{array}{r}
 1111 \quad 1111 \quad 0110 \\
 0000 \quad 0000 \quad 1001 \\
 \quad \quad \quad + 1 \\
 \hline
 0000 \quad 0000 \quad 1010 \Rightarrow 10
 \end{array}$$

(iii) PC $(\pm)$ $\underbrace{\text{imm} \times 2}_{\text{offset}}$
 ↓
 Based on
 sign bit

UJ type Instruction:



Jal x0, 2000



$$2000 = 0000 \ 0000 \ 0111 \ 1101 \ 0000$$

$$= 0 \quad \underbrace{0000 \ 0000}_{19:12} \quad \underbrace{0111 \ 1101 \ 0000}_{11 \quad 10:1}$$

20

Given a branch on register x10 being equal to zero;
`beq x10, x0, L1` replace it by a pair of instructions that offers
 a much greater branching distance.

Ans: These instructions replace the short address
 conditional branch:

`bne x10, x0, L2`

`jal x0, L1`

L2:

* Uses 20 bit immediates
 for larger jumps

* for more large jumps

(32 bit) `lui`: load address
 [31:12] to a temp reg.

`jaln`: add address [11:0]
 and jump to target

Procedure calling.

* Lets assume procedure like a spy

Execution process of spy:

(i) Leaves with a secret plan

(ii) acquires resources

(iii) perform the task.

(iv) Covers traces.

(v) returns to the point of origin with desired

answer.

Execution of procedure

Step 1: Put parameters in a place where procedure can access them. $\rightarrow (X_{10}-X_{17})$

Step 2: Transfer control to the procedure.

Step 3: Acquire storage resources needed for procedure.

Step 4: Perform the desired task

Step 5: Put the result value in a place where the calling program can access it. $\rightarrow [X_{10}, X_{11}]$

Step 6: Return control to the point of origin $\rightarrow X_1$

[A proc can be called from several points in a program]

def calc(a,b,c):

d = calc(a,b,c)

→ Decrease the stack pointer by the # locations you need

→ Then store the values in stack.

Leaf Procedure:

def leaf-example (g, h, i, j):
 f = (g+h) - (i+j)
 return f

leaf-procedure:

addi x2, x2, -24

sd x5, 16[x2]
sd x6, 8[x2]
sd x20, 0[x2] } → storing into stack

add x5, x10, x11
add x6, x12, x13
sub x20, x5, x6 }

addi x10, x20, 0 → storing return value.

ld x20, 0[x2]
ld x6, 8[x2]
ld x5, 16[x2] } loaded back

addi x2, x2, 24 → stack pointer increase

ret x0, 0[x1]

g = x10

h = x11

i = x12

j = x13

f = x20

te mp = x5, x6

already
is use
by
the
caller

